

ТЕМА: ПРОСТЫЕ АЛГОРИТМЫ СОРТИРОВКИ



ВВЕДЕНИЕ

Начиная изучение алгоритмов, следует изучить самые популярные и полезные алгоритмы, а именно - сортировка пузырьком и выбором. Именно в этой теме мы и рассмотрим 2 алгоритма

УРОК: СОРТИРОВКА ПУЗЫРЬКОМ

Сортировка пузырьком

это простой алгоритм сортировки, который многократно проходит по списку, сравнивает соседние элементы и меняет их местами, если они расположены в неправильном порядке. Этот процесс повторяется, пока список не будет отсортирован



Принцип работы

- 01 Проход по списку:** начните с первого элемента и сравните его с следующим
- 02 Обмен элементов:** если текущий элемент больше следующего, поменяйте их местами
- 03 Повторение:** повторяйте процесс для всех элементов до конца списка. Каждый проход приводит к "всплыванию" наибольшего элемента в конец списка
- 04 Оптимизация:** если в очередном проходе не было произведено обменов, список считается отсортированным

Пример кода

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break

# Пример использования:
arr = [64, 34, 25, 12, 22, 11, 90]
bubble_sort(arr)
print("Отсортированный массив:", arr)
```

Характеристики

Сложность

Временная сложность: $O(n^2)$ в худшем и среднем случае, где n — количество элементов в массиве. $O(n)$ в лучшем случае (если массив уже отсортирован)

Пространственная сложность: $O(1)$, так как сортировка выполняется на месте

Преимущества

Простота реализации

Недостатки

Низкая эффективность для больших массивов

Вывод

Мы рассмотрели пузырьковую сортировку. Эта сортировка является основой для изучения более сложных алгоритмов и структур данных. Понимание работы и сложности поможет вам в дальнейшем при разработке и оптимизации алгоритмов для более сложных задач